

v0.4.1 — Patch Status and v0.4.2 Scoping

What was amended and committed, how each open question was actually resolved, and what genuinely remains — separated into refinements that belong in v0.4.2 and new subsystems that belong in v0.5.

Repository	costinfi/aerf
Branch	claude/aerf-core-domain-model-gwumun
Head commit at issue	cc602de — “Amendment 7 + triage the remaining open-questions register”
Spec baseline	AERF v0.4 — Canonical Pre-Implementation Documentation
Patch document	docs/aerf-v0.4.1-patch.md (7 amendments + 1 non-normative note)
Implementation state	6 Maven modules · 219 tests green · 21 increments · 2 real repositories scanned

Headline for planning

AERF v0.4's own MVP freeze table (§11) is complete, and every open question that real evidence or a text ambiguity could settle has been settled — **8 of the 22 questions raised across the project are closed**. The 14 that remain were not skipped: each depends on a subsystem that was never commissioned. Five of them are cheap refinements to machinery that already exists (**v0.4.2**); nine require new subsystems that were explicitly outside the MVP from the beginning (**v0.5**). One of the five — baseline-relative drift (§5.3) — is no longer blocked at all: Increment 19 built the storage it was waiting for.

SECTION 1

Executive summary

This report covers the state of the AERF reference implementation at commit `cc602de`. It answers three questions: what the v0.4.1 patch actually contains and what was committed against it; how each resolved open question was resolved in practice; and what is genuinely still outstanding, split by whether it belongs to a v0.4.2 refinement pass or to v0.5.

The shape of the work so far

v0.4.1 is not a rewrite of v0.4. It is a set of **seven numbered amendments** to specific sections of the frozen v0.4 document, each adopted because building the reference implementation ran into something the specification either did not represent at all, or left genuinely ambiguous. Every amendment cites the original text, the problem found, the decision taken, and the increment that produced the evidence — so the record of why the model changed survives independently of the code.

Alongside the patch, an **open questions register** tracks conceptual questions that surfaced during implementation but were not resolved. The register's discipline is that an entry leaves it only when actually settled — not when merely discussed. That discipline is why the outstanding list below is trustworthy as a planning input.

22 questions raised across 21 increments	8 resolved 6 via patch amendments, 2 via implementation	14 outstanding 5 for v0.4.2, 9 for v0.5	219 tests green across 6 Maven modules
--	--	---	--

Counts as of commit `cc602de`. “Questions raised” includes four resolved before the register adopted numbering; the register itself numbers 18, of which 4 are closed.

What changed most recently

Three commits closed out the v0.4.1 line. **Increment 20** taught the extraction adapter to recognise Spring Data repositories by marker-interface inheritance rather than only by `@Repository` — the concrete failure case a real spring-petclinic run had exposed. **Increment 21** added the structural `MEMBER_OF` relation the canonical model was missing (Amendment 6), and caught a real measurement bug in the process. The **triage pass** resolved the confidence/role-outcome ambiguity (Amendment 7) and re-read every remaining register entry against the same bar, recording why each stays open.

SECTION 2

What v0.4.1 contains

Seven amendments, plus one non-normative implementation note. Six of the seven closed a question that had been formally recorded as open; Amendment 5 came from a planning decision rather than a register entry. Three amendments required no code at all — they formalise an interpretation the implementation had already adopted without ever stating it.

Amendment	Affects	Incr.	Code?
1 Evidence gains an execution-context field	§2.2, §2.4, App. A	6	Yes
2 Layer-entropy matrix must be declared explicitly, not derived from an ordering	§4.1	3	No
3 “Evidence-weighted” N+1 score is defined concretely	§4.3, App. B	6	Yes
4 Iterative role refinement is scoped to monotonic transitions	§3.2	4	Yes
5 Evidence gains structured attributes	§2.2, §2.4, App. A	11	Yes
6 Edge relation set gains a structural MEMBER_OF relation	§2.4	21	Yes
7 Confidence is unaffected by a node's role outcome	§5.4	21	No

“Code?” distinguishes an amendment that changed the implementation from one that formalises an interpretation already in force. Amendments 2 and 7 changed no code by design — a reader of v0.4 alone would not have known the restriction existed.

The two amendments a reader should not skip

Amendment 2 — the ordering is illustrative, not a derivation recipe

§4.1 offers a “typical allowed ordering” (Presentation → Application → Domain → Persistence → Infrastructure) next to a requirement that the relation matrix stay governance-configurable. Read together, that invites deriving the matrix from the ordering. Two natural derivations were tried and **both contradict a case v0.4 treats as settled elsewhere**: adjacent-layers-only flags Application → Persistence, an unremarkable pattern; any-forward-call fails to flag Presentation → Persistence, which §6.3's own worked invariant calls a critical violation. Since no mechanical rule matches the framework's own example, the matrix must always be declared explicitly. No future implementation should reintroduce a derivation silently.

Amendment 4 — termination is structural, not asserted

§3.2 states that inference “terminates when a fixed point is reached”, but establishes no such guarantee for an arbitrary refinement function F. Two rules that each flip a node's role in response to the other would oscillate forever. v0.4.1 scopes F to rules that may only fire on an Unknown node and may only return a concrete role. The count of Unknown nodes then strictly decreases on any pass that changes anything, over a finite node set — so termination within $|V|+1$ passes is guaranteed by structure. The more general revising form of F is deferred, not ruled out; it is register entry #5, and needs its own convergence argument.

Amendment 6, and the measurement bug it exposed

Adding `MEMBER_OF` — a purely structural relation from a `FUNCTION` node to the `COMPONENT` that declares it — looked inert. It was not. A `MEMBER_OF` edge's target is, by construction, always resolvable: a method's declaring class is never external. `AnalysisConfidence` (§5.4) is deliberately graph-wide and counted every edge, so these guaranteed-resolved edges inflated confidence on the pipeline's own defect sample from **0.714 to 0.857** — a movement caused entirely by the sample having methods, with no relationship to how well extraction resolved anything genuinely uncertain. Caught immediately by an existing exact-value assertion. Confidence now excludes the relation from both numerator and denominator, and a test proves the exclusion is unconditional rather than merely moot.

The wider lesson, worth carrying into v0.4.2: **any new relation type is a measurement change, not just a modelling change**. The three entropy calculators were unaffected only because each already scopes itself to an explicit relation allow-list. Confidence had no such filter, and needed one.

SECTION 3

What was committed

Twenty-one increments, each one commit, each with its own document in `docs/`. Increments 1–10 completed v0.4 §11's MVP freeze table. Increments 11–16 built the extraction adapter under a standing plan. Increments 17–21 hardened it, ran it against real code, published the results, and closed out the remaining answerable questions.

Increments 1–10 • the MVP freeze table (§11)

- | | |
|----|---|
| 1 | AERF core domain model |
| 2 | Seed role inference |
| 3 | Layer entropy |
| 4 | Iterative graph-relationship role refinement |
| 5 | Cycle entropy |
| 6 | Basic persistence / N+1 heuristic |
| 7 | Basic security entropy |
| 8 | Calibration model — aggregation, maturity, confidence |
| 9 | Invariant DSL core |
| 10 | JSON evidence / reporting |

Increments 11–16 • the ExtractionAdapter Plan

- | | |
|----|---|
| 11 | Evidence structured attributes (Amendment 5) |
| 12 | The extraction bridge — <code>aerf-extraction</code> module |
| 13 | <code>aerf-openrewrite</code> , class-level extraction |
| 14 | Method-level extraction and execution context |
| 15 | Spring / annotation semantic evidence |
| 16 | Pipeline runner and CI — plan complete |

Increments 17–21 • hardening, real code, closing out

- | | |
|----|--|
| 17 | Iterative <code>StronglyConnectedComponents</code> (closed #7) |
| 18 | A real-repository pipeline run (exposed #1's failure case) |
| 19 | Scan dashboard, published to GitHub Pages |

- 20** | Spring Data marker-interface evidence (closed #18)
- 21** | Structural MEMBER_OF relation (closed #17, Amendment 6)

Verification state

Module	Responsibility	Tests
aerf-model	Canonical domain model	32
aerf-analysis	Role inference, entropy metrics, invariants, calibration	117
aerf-report	Evidence and report serialization	24
aerf-extraction	Technology-agnostic extraction bridge	23
aerf-openrewrite	OpenRewrite adapter — the only module carrying it	17
aerf-pipeline	End-to-end runner	6
Total		219

The §9 module boundary — “the AERF graph must not expose OpenRewrite AST types” — is enforced mechanically rather than by convention: `maven-enforcer-plugin` bans `org.openrewrite:*` from every module except the adapter, so a violation fails the build.

Two real repositories have been scanned end-to-end and their reports published to the live dashboard: **spring-petclinic** (modern Spring Boot) and **spring-framework-petclinic** (pre-Boot, JSP-era). The second produced the project's first genuine persistence-entropy finding — a real N+1 in `JdbcOwnerRepositoryImpl#loadOwnersPetsAndVisits`.

SECTION 4

How the resolved questions were resolved

Eight questions are closed. What follows is how each was actually settled — in several cases not the way the question itself predicted, which is the more useful part of the record.

A1	How is “repeated execution context” represented at all? RESOLVED — Amendment 1, Increment 6 §4.3's N+1 heuristic depends on knowing whether a persistence access happens inside a loop; §2's frozen model had no field for it. Evidence gained an <code>execution_context</code> field with three values — SINGLE, ITERATED, UNKNOWN — defaulting to UNKNOWN specifically so every piece of evidence recorded in Increments 1–5 keeps its original meaning exactly. A general control-flow graph concept was considered and rejected as disproportionate to what §11 itself frames as a “basic” heuristic.
A2	Does the layer-entropy matrix derive from the stated ordering? RESOLVED — Amendment 2, Increment 3 No. Both natural derivations contradict cases v0.4 settles elsewhere (detail in Section 2). The matrix must always be declared explicitly; v0.4.1 defines no default derivation.

A3	What does “evidence-weighted” mean for the N+1 score? RESOLVED — Amendment 3 Appendix B gave the formula as “evidence-weighted N+1 patterns / relevant persistence contexts” while defining “evidence-weighted” nowhere. The weight of one relevant persistence context is now the count of its evidence items marked ITERATED — the number of independently observed repeated-execution call sites backing that access. Reported alongside the plain flagged/relevant ratio, not instead of it, per §14's measurement-before-aggregation principle.
A4	Is iterative role refinement's F guaranteed to terminate? RESOLVED — Amendment 4, Increment 4 Not for arbitrary F. Scoped to monotonic transitions, termination within $V +1$ passes becomes structural (detail in Section 2). The revising form remains open as register entry #5.
#7	Will the recursive Tarjan implementation overflow the stack on real graphs? RESOLVED — Increment 17, implementation debt (no spec change) Yes, and it was fixed before it could bite. StronglyConnectedComponents was rewritten iteratively and verified against a 200,000-node chain that reliably StackOverflowErrors the previous recursive version on a default JVM stack. This was the explicit gate on running the pipeline against a real repository — Increment 18 followed immediately.
#3	How should per-node Unknown and graph-wide confidence (§5.4) interact? RESOLVED — Amendment 7, Increment 21 triage (no code change) They don't. Confidence asks whether the graph resolved an edge's endpoints — an extraction-stage question — and never inspects a node's Role, a role-inference-stage outcome. A node can be fully resolved structurally and still be Unknown by role; conflating the two would hide which stage actually degraded. The amendment formalises what AnalysisConfidence had implemented since Increment 8 without ever stating it.
#17	Should the model have a structural FUNCTION → declaring COMPONENT relation? RESOLVED — Amendment 6, Increment 21 Yes: MEMBER_OF. The question offered three options; the first and third were adopted together and the second was rejected for a concrete reason found while evaluating it. A refinement rule consuming the relation could only resolve a method one full pass after its class resolves, because rules see roles only as of the start of a pass (Amendment 4). That would shift pass counts several already-verified worked examples depend on, for no gain over the evidence-time propagation that already works. The relation was added for its own sake; role inference was left alone.
#18	Should Spring Data repositories be recognised by marker interface, not just @Repository? RESOLVED — Increment 20, adapter-level (no spec change) Yes — but not where the question assumed. It proposed a new seed rule; that was impossible. RoleInferenceRule is contractually a pure function of one node's own evidence with no graph access, so a rule matching an external, unextracted supertype by FQN could never have been written as a seed rule at all. The adapter recognises the marker interface and emits ordinary spring-data evidence instead — the existing seed rule needed no change whatsoever. Scoped to directly declared supertypes, not transitively; no real repository has yet contradicted that choice.

Partially answered, deliberately still open — #1

“When does seed-only role classification actually fail?” got its first real answer in Increment 18: every repository interface in a real spring-petclinic checkout came back UNKNOWN, because idiomatic Spring Data carries no annotation. Increment 20 fixed that specific shape. The entry stays open because closing one named failure case does not establish there are no others — and no case has yet been found where seed and graph refinement together get a wrong answer, as opposed to seed alone missing evidence an adapter had not been taught to produce.

SECTION 5**What remains outstanding**

Fourteen entries. Every one was re-read in the Increment 21 triage against the same bar the resolved questions met, and left open deliberately. The reason is consistent: each depends on something that does not exist yet and was never commissioned. Deciding them now would mean designing a speculative subsystem against nothing.

Grouped by what each is actually blocked on

Blocked on	Entries	What it covers
Governance configuration format does not exist	#2, #4, #6	Governance evidence class (§3.3's fourth class); per-subsystem layer matrices; per-module cycle-entropy scoping. All three are the same shape: a policy an organisation declares, with no format yet to declare it in.
Governance-facing risk view is undesigned	#9, #14, #15, #16	How an approved exception suppresses a persistence finding; how §5.3's R combines with invariant violations; how E_{inv} aggregates its indicators; the unified view all three imply. §6 gives no combination formula at all, and the constraint that entropy, drift and violations stay separately visible must survive whatever is chosen.
Needs a convergence argument	#5	Non-monotonic (revising) role refinement — explicitly deferred, not ruled out, by Amendment 4. Needs a bound on how many times a single node may be revised before it can be attempted at all.
Needs a new rule shape	#10	CSRF and mass-assignment are relationship concerns, unlike the implemented XSS rule, which is a single-node check. Needs either an edge-aware security-rule variant or a documented node-evidence convention for binding/form configuration.
Blocked only on a decision someone has to want	#1, #8, #11, #12, #13	These five need no new subsystem — see Section 6. Two of them are now unblocked by work that has already shipped.

The full outstanding list

#	Question	From incr.	Proposed
1	When does seed-only role classification actually fail?	2	0.4.2
2	Rule authorship boundary for the “Governance” evidence class	2	0.5
4	Should LayerPolicy support per-subsystem matrices?	3	0.5
5	Non-monotonic (revising) role refinement	4	0.5
6	Should cycle entropy be scoped below the whole graph?	5	0.5
8	Should the N+1 heuristic constrain the source side too?	6	0.4.2
9	How does an approved exception suppress a persistence finding?	6	0.5
10	How should CSRF and mass-assignment be modelled?	7	0.5
11	Should security “concern” become a closed vocabulary?	7	0.4.2
12	How should baseline-relative drift represent and load a baseline?	8	0.4.2
13	Should confidence be graph-wide only, or per-dimension too?	8	0.4.2
14	How does the drift-aware risk model R combine with violations?	8	0.5
15	How should E_{inv} aggregate invariant indicators?	9	0.5
16	How do violations, E_{total} and drift form one governance view?	9	0.5

SECTION 6

Proposed split — v0.4.2 versus v0.5

The dividing line that matches both the project's history and the stated intent for v0.5 is **whether an entry needs a subsystem that was never part of the MVP**. Five entries do not: each is a refinement to machinery that already exists and already has tests. Nine do, and they cluster into three coherent subsystems rather than nine independent pieces of work — which makes v0.5 a much smaller-sounding project than a list of nine questions suggests.

v0.4.2 — five refinements, no new subsystems

#12 Baseline-relative drift (§5.3)

HIGH

Deferred in Increment 8 “until JSON/reporting gives it somewhere to live”. That place now exists and was built for a different reason: Increment 19's Supabase scans table already stores every historical scan with all its metrics and the full report JSON. $\Delta_d = E_d^t - E_d^0$ is a subtraction once a prior measurement can be loaded — and the dashboard would show drift per project immediately. **The blocker on this entry has already been removed; nobody noticed.**

#13 Per-dimension confidence (§5.4)

HIGH

Distinct from the now-resolved #3. Each entropy calculator already carries its own relevantRelations subset, so “persistence entropy computed at 90% confidence” is the existing computation restricted to an existing filter. Cost is mostly plumbing: a field per metric in the report, the JSON schema and the dashboard. Amendment 7's reasoning about which stage a number describes applies directly.

#8 N+1 source-side constraint (§4.3)

MEDIUM

Left open in Increment 6 pending “real evidence about whether the simpler version over- or under-flags in practice”. That evidence now exists: two real repository runs, including one genuine N+1 finding in the legacy PetClinic. The question can finally be decided against data rather than intuition.

#11 Security concern vocabulary (§4.4)

LOW

Currently free-text by deliberate design. Documented string constants — not necessarily an enum — would let an invariant reference a concern reliably. Cheap, and worth doing before invariants start depending on the strings informally.

#1 When does seed-only classification fail?

ONGOING

Not a build task: an empirical question that each new real-repository scan answers a little more. The dashboard already collects the evidence. Worth keeping open and revisiting rather than scheduling.

If v0.4.2 does only one thing, make it #12

Drift is the one metric family v0.4 specifies (§5.3) that the implementation has never produced a single number for — not because it is hard, but because in Increment 8 there was nowhere to keep a baseline. Increment 19 built exactly that store, for the dashboard, without the two being connected. Wiring them together turns a static per-scan report into the trend view the framework was always aiming at, and it unblocks the R side of #14 for v0.5 as a side effect.

v0.5 — three subsystems, not nine questions

The nine remaining entries are not nine projects. They resolve into three subsystems plus two independents, and the dependency order between them is fairly strict.

A

Governance configuration

#2, #4, #6 (+ enables #9, #11)

The largest and the one everything else waits on. §3.3 names Governance as one of four evidence classes and nothing represents it; §4.1 requires the layer matrix to be governance-declared; §4.2's global cycle scope wants per-module policy. One format, declared once, serves all three — and gives #9's exception model and #11's concern vocabulary somewhere to live. **Do this first.**

B

Governance-facing risk view

#14, #15, #16 (+ #9)

How E_{total} , drift and invariant violations are presented together without collapsing into a single “architecture score” — a constraint the project has held throughout. Needs §6.1's λ_k aggregation decided (#15) and R's invariant term defined (#14). Depends on A for policy input and benefits from v0.4.2's #12 for the drift term.

C

Extended security modelling

#10

CSRF and mass-assignment need an edge-aware security rule, parallel to `GraphRoleRefinementRule`, or a node-evidence convention for binding configuration. Independent of A and B; can run in parallel.

D

Non-monotonic role refinement

#5

Independent, and gated on a convergence argument rather than on any other subsystem. Worth attempting only if a real codebase produces a case where a weakly-seeded node surrounded by strongly-classified neighbours is genuinely misclassified — which #1's ongoing evidence collection would surface.

Suggested order

v0.4.2: #12 (drift, wire the existing store) → #13 (per-dimension confidence) → #8 (N+1 source side, now decidable against real data) → #11 (concern constants). #1 continues in the background with each new scan.

v0.5: A (governance configuration) first, since B and half of A's own siblings depend on it → B (risk view) once A and v0.4.2's drift work land → C and D in parallel, neither blocking anything else.

One caution carried forward from Amendment 6

Every one of these adds either a relation type, a metric, or a policy input. Amendment 6 showed that adding a relation silently moved a calibration number that had nothing to do with it, because one metric had no relevance filter while three others did. Before landing anything in v0.4.2 or v0.5, check what it changes in `AnalysisConfidence` and in each calculator's `relevantRelations` — the exact-value assertions in the pipeline tests are what caught it last time, and they are worth keeping exact.

APPENDIX A

Commit trail

The commits that carry the v0.4.1 patch and the increments referenced above, newest first. Every increment is one commit with its own document under *docs/*.

Commit	Message	
cc602de	Amendment 7 + triage the remaining open-questions register	PATCH
42fcdd	Increment 21: structural MEMBER_OF relation (Amendment 6)	PATCH
129e40d	Increment 20: recognize Spring Data repositories by marker interface	
13ebb48	Add a second real scan: spring-framework-petclinic (pre-Boot, JSP-era)	
3c4114a	Add info-icon tooltips explaining each metric on the dashboard	
2415564	Fix CI/Pages workflows: this repo has no main branch	
6fd21a3	Increment 19: scan dashboard, published to GitHub Pages	
a64819e	Increment 18: run the pipeline against a real cloned repository	
6456c93	Increment 17: rewrite StronglyConnectedComponents iteratively	
fc07d44	Increment 16: pipeline runner and CI — ExtractionAdapter Plan complete	
9051424	Increment 15: Spring/annotation semantic evidence	
ca4a245	Increment 14: method-level extraction and execution context	
953fedd	Increment 13: aerf-openrewrite class-level extraction	
88da46a	Add the extraction bridge: aerf-extraction module (Increment 12)	
77047ec	Add Evidence structured attributes (Increment 11)	PATCH
a3f4d5a	Add ExtractionAdapter Plan for Increments 11-16	
(various)	Increments 1-10 — one commit each, see docs/increment-01 ... increment-10	
a67ff19	Adopt AERF v0.4.1 patch: formalize amendments (1-4)	PATCH

“PATCH” marks a commit that added or changed an amendment in docs/aerf-v0.4.1-patch.md. All commits are on branch `claude/aerf-core-domain-model-gwumun`, which is also this repository's default branch.

Source documents in the repository

Path	Contents
docs/aerf-v0.4.1-patch.md	The seven amendments, each with original text, problem, decision, evidence
docs/open-questions-register.md	Living register — outstanding entries plus the resolved list, with the Increment 21 triage note
docs/extraction-adapter-plan.md	The standing plan covering Increments 11-16, amended by appending
docs/increment-NN-*.md	One document per increment, 21 in total

Report generated from the repository state at commit cc602de. Every figure in this document — test counts, confidence values, question counts — is taken from the committed documents and a full `mvn -B test` run, not from recollection.